

coop-writing v1.5.5

Quick Reference (EN-US)

Geraldo Xexéo

Purpose

`coop-writing` is a LaTeX front end for the editorial life cycle of a document. Editorial information stays in the source instead of depending on an editing interface such as Overleaf. The same project can therefore be edited in Overleaf, locally, or through Git while comments, proposed changes, drafts, anonymization, TODOs, and publication behavior remain controlled by LaTeX.

UTF-8 and engines

All project sources should be UTF-8. Current LaTeX uses UTF-8 as its default input encoding, so this example does not load `inputenc`. With pdfLaTeX it selects T1 font encoding; LuaLaTeX uses its native Unicode model.

Explicit engine detection and regression coverage for pdfLaTeX and LuaLaTeX are tracked in issue #34.

Start here

For normal collaborative writing:

```
\usepackage[editing]{coop-writing}
\cwnamedef{alice}{blue}{Alice}
\cwnamedef{bob}{red}{Bob}
```

Then write, for example:

This claim `\alice{Check the evidence.}` needs revision.

```
\alice[This sentence]{Please simplify the wording.}
```

```
\bobsug{This sentence is a proposed insertion.}
```

```
\bobswap{old wording}{new wording}
```

Editorial modes

Use one main mode per compilation.

<code>editing</code>	Working mode. Shows comments, suggestions, TODOs, subjects, and drafts. Material that would be anonymized is marked rather than hidden.
----------------------	---

<code>submit</code>	Submission mode. Hides editorial marks and anonymizes protected material by default.
<code>publish</code>	Clean publication mode. Hides editorial marks, does not anonymize, and keeps the old side of unresolved suggestions.
<code>acceptingpublish</code>	Clean publication mode that globally accepts suggestions and replacements.

Use `noanonymize` together with `submit` when a submission is not blind. Lower-level feature options also exist: `comments`, `suggestions`, `subjects`, `drafts`, `todos`, `anonymize`, and `changecolor`. Prefer a main editorial mode for ordinary projects.

A future `key=value` configuration interface is proposed in issues #32, #33, and #46.

Collaborators

```
\cwnamedef{alice}{blue}{Alice}
```

For each collaborator stem `name`, `coop-writing` creates:

<code>\name{comment}</code>	Comment without selected text.
<code>\name[selection]{comment}</code>	Comment attached to highlighted text.
<code>\name</code>	Labeled comment.
<code>r[selection]{comment}{label}</code>	
<code>\name x[...]</code>	Struck-through comment.
<code>\name rx[...]</code>	Labeled struck-through comment.
<code>\name sug[comment]{new}</code>	Proposed insertion.
<code>\name rem[comment]{old}</code>	Proposed removal.
<code>\name swap[comment]{old}{new}</code>	Proposed replacement.
<code>\name troca[comment]{old}{new}</code>	Portuguese synonym of <code>swap</code> .

Two collaborator families are predefined: `\cwauthor` and `\cweditor`.

Use `\cwsetcommwarn{symbol}` to place a symbol before comment marks.

TODOs

The intended default is a short editorial comment:

```
\todo{Check this argument.}
```

The large inline box is explicitly opt-in:

```
\todo[inline]{Rewrite this paragraph before submission.}
```

Its background can be changed with `\cwdefinetodocolor`. Issue #45 tracks the regression test and implementation cleanup for this distinction.

Citation needed

```
This quantitative claim needs a source.\pleasecite
```

```
Another claim.\pleasecite[prefer a recent systematic review]
```

Draft blocks

```
\begin{cwdraft}[Alternative explanation]
This paragraph is deliberately provisional.
\end{cwdraft}
```

```
\cwsetdraftcolor{cyan}
```

Draft material is visible while editing and suppressed in clean modes.

Anonymization

```
\cwanon{University of Example}
\cwanoncite{key}
\cwanoncitem[chap.~2]{key}
\cwanoncitett{key}
```

Customize replacements with:

```
\cwsetanoncolor{violet}
\cwdefanonntext{Anonymous institution}
\cwdefanoncitetttext{(Anonymous, Year)}
\cwdefanonciteptext{(Anonymous, Year)}
\cwdefanoncitetttext{Anonymous (Year)}
```

For repeated named material:

```
\cwblind{\myinstitution}
    {Federal University of Example}
    {Anonymous institution}
```

Subjects and main ideas

```
\cwsubject{Motivation}
This paragraph explains why the work is necessary.
```

```
\cwmain{The package separates editorial state from document content.}
The rest of the paragraph supports that main idea.
```

```
\cwsetsubjectcolor{cyan}
\cwsetmaincolor{yellow}
```

The rendering of `\cwmain` can be customized by redefining `\cwmainemphasis`.

Mode-dependent files

```
\cwinput{editing-version}{submit-version}{publish-version}
\cwinclude{editing-chapter}{submit-chapter}{publish-chapter}

\cwinputediting{editing-only}
\cwinputsubmit{submit-only}
\cwinputpublish{publish-only}
```

```
\cwincludedeediting{editing-only-chapter}  
\cwincludesubmit{submit-only-chapter}  
\cwincludepublish{publish-only-chapter}
```

Lists

```
\listofcomments  
\listofcitationneeds  
\listofsubjects
```

The planned unified report is described in issue #49.

Text color helpers

In editing mode:

```
\cwcolor{blue}  
This text is blue.  
\cwrestorecolor
```

The low-level `\cwsavecolor` helper is also available. Issue #54 proposes making every package color configurable from one user-facing interface.

Portuguese aliases

Compatibility aliases include `\cwautor`, `\cwassunto`, `\favorcitar`, and `\listofassunto`, together with Portuguese package options. The Portuguese `rascunho` environment alias is retained for compatibility; issue #40 tracks the broader internationalization cleanup.

Known v1.5.5 limitations

- `tocloft` is a major compatibility risk; removal is proposed in #36.
- `hyperref` is currently loaded automatically in normal configurations; making it optional and sanitizing PDF strings is proposed in #37.
- Moving arguments/headings need explicit safe behavior; see #26.
- Generated copies in the repository have diverged historically; see #43.
- A formal compatibility matrix for standard, publisher, UFRJ, COPPE, and Poli classes is proposed in #55.

Recommended workflow

Use `editing` while writing and `submit` for a blind submission when required. After editorial decisions are resolved, use `publish`; use `acceptingpublish` when all proposed changes should be accepted. Keep the source as the authoritative record of the editorial process.